Getting of attributes in JavaScript

getAttribute method

The getAttribute method reads the value of a specified attribute on a tag.

Syntax

```
element.getAttribute(attribute name);
```

Example

Let's output the content of the value attribute of an element:

```
<input id="elem" value="abcde">
```

```
let elem = document.querySelector('#elem');
let value = elem.getAttribute('value');

console.log(value);
```

The code execution result:

```
'abcde'
```

№1

⊗jsPmAtGt

Given an element:

```
<input id="elem" value="text">
```

Get the value of its value attribute.

№2

⊗jsPmAtGt

Given an element:

```
<input id="elem" class="www zzz">
```

Get the value of its class attribute.

Setting of attributes in JavaScript

setAttribute method

The setAttribute method allows you to change the value of a given attribute of a tag.

Syntax

```
element.setAttribute(attribute name, new value);
```

Example

Let's change the value of value attribute of an element:

```
<input id="elem" value="abcde">
```

```
let elem = document.querySelector('#elem');
elem.setAttribute('value', '!!!!');
```

The HTML code will look like this:

```
<input id="elem" value="!!!!">
```

№1

⊗jsPmAtStM

Given an element:

```
<input id="elem">
```

Set its value attribute to 'text'.

№2

⊗jsPmAtStM

Given an element:

```
<input id="elem">
```

Set its class attribute to 'valid'.

Removing of attributes in JavaScript

removeAttribute method

The removeAttribute method removes the given attribute from some tag.

Syntax

```
element.removeAttribute(attribute name);
```

Example

Let's remove the value attribute from an element:

```
<input id="elem" value="abcde">
```

```
let elem = document.querySelector('#elem');
elem.removeAttribute('value');
```

The HTML code will look like this (the value attribute will disappear):

```
<input id="elem">
```

№1

⊗jsPmAtRm

Given an element:

```
<input id="elem" value="text">
```

Remove the value attribute from it.

Attributes checking in JavaScript

hasAttribute method

The hasAttribute method checks if an element has a given attribute. If the attribute is present, it will output true, if not, it will output false.

Syntax

```
element.hasAttribute(attribute name);
```

Example

Let's check for the presence of the value attribute on an element:

```
<input id="elem" value="abcde">
```

```
let elem = document.querySelector('#elem');
console.log(elem.hasAttribute('value'));
```

The code execution result:

```
true
```

Example

And now there is no the value attribute:

```
<input id="elem">
```

```
let elem = document.querySelector('#elem');
console.log(elem.hasAttribute('value'));
```

The code execution result:

```
false
```

№1

⊗jsPmAtCh

Given an element:

```
<input id="elem" value="text">
```

Check if it has the value attribute.

Custom attributes in JavaScript

HTML allows you to add your own custom attributes to tags. Such attributes should start with data- followed by whatever attribute name you like.

Custom attributes can be used in a huge number of different ways. We will explore many of these methods later in the tutorial, and there are many more you can invent yourself later on.

Access to such attributes is not arranged in a standard way. You can't just refer to the element's property of the same name, as we did before, but you should use the special property dataset, after which the attribute name is written using a dot without data-. For example, if our attribute is called data-test, then to refer to it we will write elem.dataset.test, where elem is a variable with our element.

Let's look at an example. Let's say we have the following element:

```
<div id="elem" data-num="1000"></div>
```

Let's display the value of its data-num attribute:

```
let elem = document.querySelector('#elem');
console.log(elem.dataset.num); // shows 1000
```

Now let's assign this attribute a different value:

```
let elem = document.querySelector('#elem');
elem.dataset.num = 123;
```

№1

⊗jsPmAtDt

Given the following code:

```
<div id="elem" data-text="str">text</div>
```

Make it so that when you click on the div, the content of its data-text attribute is added to the end of its text.

№2

⊗jsPmAtDt

Given divs that contain their ordinal number in the data-num attribute:

```
<div data-num="1">text</div>
<div data-num="2">text</div>
<div data-num="3">text</div>
<div data-num="4">text</div>
<div data-num="5">text</div>
```

Make it so that when you click on any of the divs, its ordinal number is written to the end.

№3

⊗jsPmAtDt

Given a button. Make it so that this button counts the number of clicks on it, writing them to some custom attribute. Let on click on another button on the screen displays how many clicks were done on the first button.

№4

⊗jsPmAtDt

Given an input:

```
<input id="elem" data-length="5">
```

In this input, the data-length attribute contains the number of characters to be entered into the input. Make it so that when focus is lost, if the number of characters entered does not match the specified one, a message is displayed about this.

№5

⊗jsPmAtDt

Given an input:

```
<input id="elem" data-min="5" data-max="10">
```

In this input, the attributes data-min and data-max contain a range. Make it so that when focus is lost, if the number of characters entered does not fall within this range, a message is displayed about this.

Hyphenated names of attributes in JavaScript

Custom attributes can contain hyphens in their names, like this:

```
<div id="elem" data-my-test="1000"></div>
```

To refer to such attributes, use camelCase:

```
let elem = document.querySelector('#elem');
console.log(elem.dataset.myTest);
```

№1

⊗jsPmAtDHN

Given the following code:

```
<div id="elem" data-product-price="1000" data-product-amount="5">
  apples
</div>
```

Make it so that when you click on the div, the purchase price is added to the end of its text, equal to the price multiplied by the quantity.

Accessing to attributes via methods in JavaScript

Custom attributes can also be accessed using methods like getAttribute, in which case the full name of the attribute should be written:

```
<div id="elem" data-num="1000" data-my-num="2000"></div>
```

```
let elem = document.querySelector('#elem');

console.log(elem.getAttribute('data-num'));    // shows 1000
console.log(elem.getAttribute('data-my-num')); // shows 2000
```

№1

⊗jsPmAtDM

Given paragraphs. Go through them in a loop and for each paragraph write down the ordinal number of this paragraph into the data-num attribute. Use the setAttribute method.

Working with array of CSS classes in JavaScript

classList property

The classList property contains pseudo-array of element CSS classes, and also allows you to add and remove element classes, check for a specific class among element classes.

We are talking about the class attribute, inside which you can write several classes separated by a space, for example www ggg zzz. With classList you can remove, for example, the class ggg without affecting other classes.

Syntax

```
element.classList;
```

Example . Number of classes

Finding the number of element classes:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');

let length = elem.classList.length;
console.log(length);
```

The code execution result:

```
3
```

Example . Looping through classes

Let's derive element classes one by one:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');
let classNames = elem.classList;

for (let className of classNames) {
  console.log(className);
}
```

The code execution result:

```
'www'
'ggg'
'zzz'
```

№1

⊗jsPmAtCAr

Given an element:

```
<p id="elem" class="www ggg zzz"></p>
```

Find out the number of its classes.

№2

⊗jsPmAtCAr

Given an element:

```
<p id="elem" class="www ggg zzz"></p>
```

Loop through its classes.

Adding of CSS classes in JavaScript

add method of the classList object

The add method of the classList object allows you to add CSS classes to an element.

Syntax

```
element.classList.add(class);
```

Example

Let's add the class kkk to an element:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');
elem.classList.add('kkk');
```

The code execution result:

```
<p id="elem" class="www ggg zzz kkk"></p>
```

Example

Let's add the class zzz to an element, which is already in the element - nothing will happen, since duplicate classes are not added:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');
elem.classList.add('zzz');
```

The code execution result:

```
<p id="elem" class="www ggg zzz"></p>
```

№1

⊗jsPmAtCAd

Given an element:

```
<p id="elem" class="www ggg zzz"></p>
```

Add class xxx to it.

Removing of CSS classes in JavaScript

remove method of the classList object

The remove method of the classList object removes a given CSS class of an element.

Syntax

```
element.classList.remove(class);
```

Example

We remove the class ggg:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');
elem.classList.remove('ggg');
```

The code execution result:

```
<p id="elem" class="www zzz"></p>
```

№1

⊗jsPmAtCR

Given an element:

```
<p id="elem" class="www ggg zzz"></p>
```

Remove the class www and the class zzz from it.

Checking of CSS classes in JavaScript

contains method of the classList object

The contains method of the classList object checks for the presence of an element's CSS class.

Syntax

```
element.classList.contains(class);
```

Example

We check if an element has the class ggg:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');

let contains = elem.classList.contains('ggg');
console.log(contains);
```

The code execution result:

```
true
```

№1

⊗jsPmAtCCh

Given an element:

```
<p id="elem" class="www ggg zzz"></p>
```

Check if it has the class ggg.

Toggling of CSS classes in JavaScript

toggle method of the classList object

The toggle method of the classList object toggles a given CSS class of an element: adds the class if it doesn't exist and removes it if it does.

Syntax

```
element.classList.toggle(class);
```

Example

In this example, when using the toggle method, the class zzz will be removed, since it is already in the element:

```
<p id="elem" class="www ggg zzz"></p>
```

```
let elem = document.querySelector('#elem');
elem.classList.toggle('zzz');
```

The code execution result:

```
<p id="elem" class="www ggg"></p>
```

Example

In this example, when using the toggle method, the class zzz will be added, since it is not present in the element:

```
<p id="elem" class="www ggg"></p>
```

```
let elem = document.querySelector('#elem');
elem.classList.toggle('zzz');
```

The code execution result:

```
<p id="elem" class="www ggg zzz"></p>
```

⊗jsPmAtCT

№1

Given an element. Add the class www to it if it doesn't exist and remove it if it does.